

BARouter: A Budget-adaptive Online Large Language Model Router Framework

Lingkai Zu
zulk2024@shanghaitech.edu.cn
ShanghaiTech University
Shanghai, China

Xiyue Peng
pengxy2024@shanghaitech.edu.cn
ShanghaiTech University
Shanghai, China

Xin Liu*
liuxin7@shanghaitech.edu.cn
ShanghaiTech University
Shanghai, China

Abstract

With the rapid advancement of large language models (LLMs), a diverse ecosystem of models with different scales and domain specializations has emerged, including LLM-based web agents and online multi-LLM server systems. LLM routing, which opportunistically leverages this diversity to balance response quality and computational cost, has become a central problem in optimizing the performance of LLM serving systems. We propose the **Budget-Adaptive Router** (BARouter), a budget-adaptive online routing framework for LLM serving systems. BARouter dynamically adjusts its routing policy for incoming queries based on the estimated response quality, query costs, and real-time budget consumption, enabling seamless adaptation to varying initial budgets and evolving input distributions without manual hyperparameter tuning. Theoretically, we prove that BARouter achieves sublinear regret over the time horizon T . The extensive experiments show that BARouter effectively allocates the right budget to the queries at the right time, consistently outperforming baseline algorithms, and remains robust to varying budget levels and shifting query distributions.

CCS Concepts

• **Computing methodologies** → *Online learning settings*.

Keywords

Large language model serving, online routing, budget efficiency

ACM Reference Format:

Lingkai Zu, Xiyue Peng, and Xin Liu. 2026. BARouter: A Budget-adaptive Online Large Language Model Router Framework. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792725>

1 Introduction

Large Language Models (LLMs) have achieved remarkable progress in recent years, demonstrating exceptional capabilities across a diverse range of tasks [2, 9, 10, 17, 29], including web tasks such as booking hotels or tickets [19, 33]. These models are predominantly guided by the Scaling Laws, relying on massive models to ensure exceptional performance. This paradigm implies that deploying and utilizing such LLMs typically demands substantial

computational resources and incurs significant inference overhead. While these powerful general-purpose models, especially when augmented with sophisticated reasoning techniques (e.g., Chain-of-Thought (CoT)), can tackle numerous challenging problems, in practice, many simple daily queries do not necessitate such formidable LLMs and complex reasoning strategies. For example, when solving a complex integral problem, it is worthwhile to employ CoT to ensure that the LLM engages in careful deliberation and precise computation. Conversely, if the objective is to query the spelling of a specific interface during programming, to minimize token consumption and enable rapid responses from the LLM, the use of CoT is unnecessary. Therefore, an alternative approach to enhancing language model performance is specialization, which develops domain-specific LLMs with a smaller size to beat the larger models within specific domains. For example, CodeQwen 7B [3] has comparable code generation capabilities with GPT-4 [20]. For programming tasks, invoking CodeQwen can achieve comparable outcomes while incurring lower costs than GPT-4. Apollo [31], a lightweight multilingual medical LLM, achieves the best performance in the medical domain, which means that assigning medical questions to Apollo can return superior response quality with low costs. Although such specialized models demonstrate strong performance on domain-specific tasks, the queries encountered in real-world scenarios typically exhibit substantial diversity. Neither large-scale general-purpose LLMs nor specialized models alone can efficiently accommodate such diverse prompts [15, 25]. Consequently, routing mechanisms have been increasingly adopted to enable cost-effective and responsive LLM serving [7, 26, 27]. Intuitively, LLM routing involves assigning user-input prompts to the most suitable LLMs, thereby achieving high response quality with minimal costs. By intelligently allocating users' prompts to appropriately scaled or domain-specific LLMs, the router enables the LLM server infrastructure to fulfill users' demands most cost-effectively.

Table 1: Existing LLM routing method with its budget/cost control strategies and feedback types.

Method	Pre-training	Budget/Cost Control	Feedback Type
[15, 32]	Not required	Quality only	Bandit
[24]	Not required	Non-adaptive	Bandit
[4]	Required	Quality only	Full
[30]	Required	Non-adaptive	Bandit
[25]	Required	Non-adaptive	Preference
[14, 18, 28]	Required	Non-adaptive	Full
BARouter(ours)	Not required	Adaptive	Bandit

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792725>

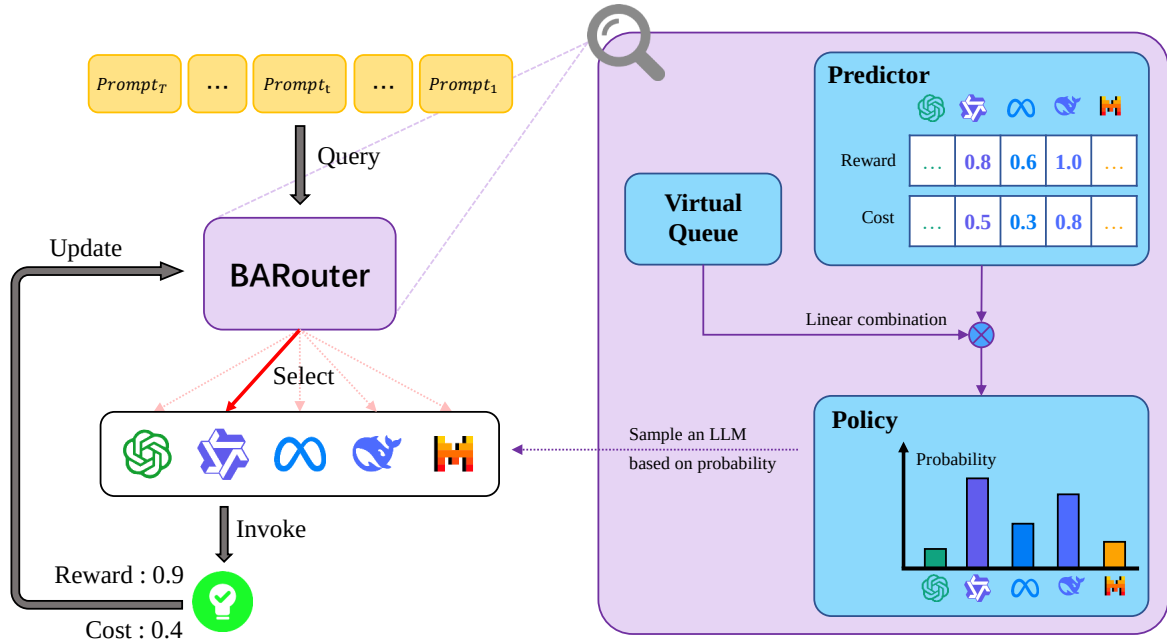


Figure 1: The pipeline of Budget-Adaptive Router (BARouter) framework. Left (workflow of the online LLM router server): Upon receiving a real-time user query/prompt, BARouter selects a suitable LLM to invoke, and the selected LLM generates the corresponding response. The observed reward and cost associated with this response are then utilized to iteratively refine subsequent routing decisions. Right (BARouter’s internal policy generation mechanism): For each incoming query, the router computes a linear combination of the estimated reward and cost guided by the virtual queue, and subsequently derives the routing policy using the EXP3 technique.

The initial studies on LLM routing mainly focus on assigning user queries from various categories (e.g., mathematics, programming, medicine, etc.) to the most suitable LLM, with the aim of solely achieving optimal response quality [4, 15, 32]. However, such (cost-unaware) routing strategies may allocate the majority of queries to the LLMs with the highest capabilities, incurring substantially high costs. A line of subsequent work has incorporated costs or budgets into the routing design [4, 14, 18, 28]. These studies formulate this as maximizing a unified objective function $u(x, a) = f(x, a) - \lambda g(x, a)$ [14, 18] or a weighted average function $u(x, a) = (1 - \mu)f(x, a) - \mu g(x, a)$ [28], where $f(x, a)$ and $g(x, a)$ denote the response quality (e.g. accuracy) and cost (e.g. API expense) for LLM a on a given query x , and $\lambda \geq 0$ or $0 \leq \mu \leq 1$ quantify user’s cost sensitivity. These approaches [4, 14, 18, 28] balance the quality and cost consumption by searching an optimal cost-sensitivity parameter μ or λ under the assumption of perfect knowledge of the environment or dataset (e.g., the query distribution and the exact f and g values), making them inherently offline and non-adaptive. Intuitively, when incoming prompts, reward/cost functions or budget limits deviate from the training distribution, these approaches are prone to out-of-distribution (OOD) issues. It is worth noting that **RouterBench** [14] and **SPROUT** [28] provide two important and comprehensive datasets for LLM routing research. These datasets are widely used benchmarks that include detailed annotations of

response quality and token cost under varying budget constraints. Besides, most existing studies rely on offline datasets with full feedback on every pair of prompt and LLM to train the routing policy [4, 13, 18, 21, 28], which is too expensive to design such datasets. An improved approach is to use preference-based feedback for training the router [25], but it still requires additional unselected LLMs’ feedback. In practical LLM serving, only the response quality and cost of the selected LLM can be observed, i.e., **bandit feedback**. The bandit feedback fundamentally differs from the full-feedback setting, where quality and cost of unselected LLMs remain unobservable during routing [15, 24, 30, 32]. The bandit-feedback, also named as “observational feedback” in Tsiourvas et al. [30], is the most aligned with real-world scenarios. Table 1 summarizes the aforementioned methods and categorizes them according to their cost-control strategies and feedback types. It can be observed that existing LLM routing approaches either lack adaptability to online and dynamic settings or require strong feedback, making them difficult to deploy in practice. Hence, a natural open question is raised:

Can we design an online budget-adaptive LLM routing framework with bandit feedback?

In this paper, we provide a positive answer to this question by introducing the **Budget-Adaptive Router (BARouter)**, an adaptive online LLM router framework. As illustrated in Figure 1, for

each real-time user query/prompt, BARouter dynamically computes the utility function by forming a linear combination of the estimated rewards and costs, weighted by a budget-adaptive virtual queue. Subsequently, BARouter translates the utility function into the routing probability using the exponential-weighting principle to balance exploration and exploitation (EXP3). The virtual queue, automatically adjusted based on the current budget and historical budget consumption, enables BARouter to make informed decisions under budget constraints. Meanwhile, the EXP3 design establishes a good balance between exploration and exploitation such that it can adapt to bandit feedback with time-varying incoming prompts. Overall, BARouter distinguishes itself from the methods in Table 1 through the following key characteristics:

- **Online Routing with Bandit-feedback.** As an online LLM router framework, BARouter does not require pre-training and can achieve LLM routing under online bandit-feedback conditions. This setup better reflects real-world scenarios, as obtaining feedback from every available large language model for a user's query is often costly or infeasible. Hence, BARouter does not incur additional overhead for collecting training data. In scenarios with insufficient data (e.g., involving unseen LLMs or queries pertaining to unseen domains), the predictor may encounter challenges in generating accurate estimation. It addresses this limitation by engaging in exploration to acquire corresponding feedback, thereby enhancing the predictor's accuracy.
- **Optimal Budget Adaptation.** BARouter is capable of accurately allocating budgets to achieve near-optimal query response performance under various budget limits, without requiring any prior information beyond the number of incoming tasks and the initial budget. This is demonstrated on two comprehensive datasets, RouterBench [14] and SPROUT [28], as shown in Figure 2. Assuming perfect knowledge of incoming queries and their corresponding quality and cost functions, we search for the optimal cost-sensitivity parameters μ or λ in [14, 28] under different budget limits, which serve as theoretical upper bounds. BARouter consistently matches these upper bounds across all budget settings.
- **Adaptation to Dynamic Environment.** BARouter operates fully online and requires minimal hyperparameter tuning, continuously adapting to shifting query distributions while providing precise and stable budget control in dynamic environments. This online adaptability makes BARouter respond to distributional changes in real time, preventing premature budget exhaustion and avoiding overly conservative routing policies as the environment shifts.

BARouter is justified with theoretical analysis and extensive empirical validation. Theoretically, we prove that BARouter achieves a sublinear regret bound of $\tilde{O}(T^{\frac{3}{4}}U^{\frac{1}{4}})$, where T is the number of incoming prompts (time horizon) and U relates to the reward/cost estimation errors. Empirically, we demonstrate that BARouter consistently adapts to varying budget constraints and distribution shifts, achieving superior performance across settings compared to baseline methods. It also exhibits strong robustness across different predictors even with coarse-grained parameter tuning.

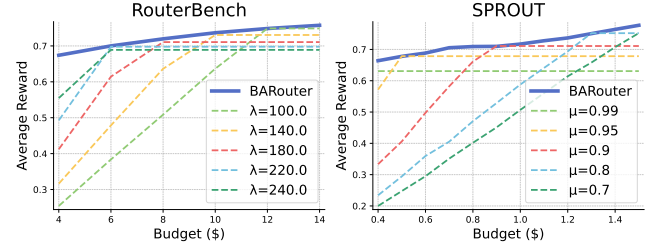


Figure 2: BARouter achieve optimal budget adaptation justified by two comprehensive datasets RouterBench [14] and SPROUT [28].

2 Related Work

LLM routing methods have been extensively studied. This section solely discusses the content most relevant to our approach.

LLM Router without Cost. A straightforward routing approach is simply selecting the LLM with the highest response quality. Chen et al. [4] Fine-tunes a model that learning problems and LLM embeddings using contrastive learning. However, this method necessitates substantial computational resources for training and relies on a large-scale dataset. In contrast, Hu et al. [16], Xia et al. [32] formulate the routing problem as a contextual bandit problem, enabling online learning of the relationship between prompts and available LLMs, thereby reducing the dependence on computational resources and datasets. Such routing strategy overlook the costs associated with invoking LLMs, which may lead to increased substantial expenses.

Cost-aware LLM Router. Jitkrittum et al. [18] and Mohammadshahi et al. [22] incorporate the average cost of LLMs into their routing strategies while predicting LLM response quality for prompts, aiming to balance the quality and cost. Note that the cost is also associated with the prompt, Somerstep et al. [28] and Mei et al. [21] train a cost estimator with a structure similar to that of the quality estimator, aiming to more accurately estimate the expense of invoking LLMs. These methods typically introduce a utility function $u = r - \lambda c$, where r and c represent the corresponding quality and cost, respectively, and λ is a predefined and fixed parameter indicating cost sensitivity. Tsiourvas et al. [30] defines set of a series λ_s ($\Lambda = \{\lambda_1, \dots, \lambda_n\}$) and predict the utility function for each λ . For $\lambda \notin \Lambda$, they use the corresponding joint network trained to generalize across the cost sensitivity. Hari and Thomson [13] incorporate costs or user-defined constraints along with response quality into the loss function during training, with the objective of developing a routing strategy that balances response quality and cost. These methods are capable of accounting for the costs associated with invoking LLMs, thereby balancing response quality and cost. However, they are all static offline routing algorithms that require pretraining or parameter tuning. Such routing approaches are highly dependent on the distribution of the training dataset and require fine-tuning hyperparameters for different budget constraints, making them less adaptable to practical online settings. To design a routing policy for practical online settings, Zhao et al. [34] selects the LLM that is expected to achieve the highest response quality while complying with the current budget constraints. However, this

method is typically over conservative such that a significant portion of the budget remains unused. When confronted with previously unseen available LLMs or user queries from entirely new domains, these methods often fail to produce accurate estimates, leading to performance degradation. Our proposed method takes the budget as input and dynamically regulates the budget to ensure the quality of LLM responses while avoiding premature budget exhaustion.

LLM Router with Different Types of Feedback. Existing LLM routing method typically necessitate a "full feedback" dataset for training [18, 21, 22, 28]. Due to constraints in computational resources and time, obtaining feedback from all available LLMs for a specific query is often costly or impractical. To reduce dependence on such datasets, Nguyen et al. [24], Zhao et al. [34] employ a preference dataset, but it still requires additional unselected LLMs' feedback. Furthermore, in real-world deployment scenarios, users are often unwilling to carefully read and compare two responses to a single query in order to indicate their preferences. This poses significant challenges for capturing user preferences during the deployment phase. Actually, without introducing additional expense, only the response quality and cost of the selected LLM can be observed in practical deployment. Tsiourvas et al. [30] utilizes the observational dataset, which is essentially the same as bandit-feedback. A direct method utilizing bandit-feedback is to model the LLM routing problem as an online multi-armed bandit problem, treating the available LLMs as arms and the response quality as the reward. Dai et al. [5] approaches LLM routing via a (non-contextual) multi-armed bandit formulation but overlooks query semantics. Nguyen et al. [24] employs a contextual bandit framework but still relies on the static utility function mentioned above, which neglects the dynamic impact of budget constraints on router decisions. Unlike their methods, we model the LLM routing problem as a contextual bandit problem *with knapsack constraints*. Consequently, our proposed online routing strategy enables the router to dynamically adapt its routing decisions based on budget constraints and evolving data distribution, explicitly guided by the semantic content of each query.

3 Problem Formulation

LLM Routing Problem. We now formally introduce the LLM Routing Problem, which is a sequential interaction process between the user's prompts and a router over $T \in \mathbb{N}^+$ rounds with a total initial budget B . During online routing, we consider a fixed set of M LLMs, denoted by $\mathcal{A} = [M] = \{1, \dots, M\}$ and let $\mathcal{X}$ be the set of all possible queries. When invoking a specific LLM $a \in \mathcal{A}$ for a query $x \in \mathcal{X}$, the generation can be sampled by $y \sim \mathbb{P}_a(\cdot | x)$, where the $\mathbb{P}_a(\cdot | x) \in \Delta(\mathcal{Y})$ is the conditional distribution over the response set $\mathcal{Y}$ for any model $a \in \mathcal{A}$. The quality of response y is $\text{Quality} : \mathcal{X} \times \mathcal{Y} \rightarrow [0, F]$ and the cost of generating response y is $\text{Cost} : \mathcal{A} \times \mathcal{Y} \rightarrow [0, G]$, where F and G are the upper bounds of the quality and cost. Since the generation y samples from $\mathbb{P}_a(\cdot | x)$ with model a , i.e. $y \sim \mathbb{P}_a(\cdot | x)$, we define the random variables $\mathcal{R}(x, a) = \text{Quality}(x, y) \in [0, F]$ and $\mathcal{C}(x, a) = \text{Cost}(x, y) \in [0, G]$ to simplify the characterization of the model reward(response quality) and cost. At each round t , the router receives a query $x_t \in \mathcal{X}$ from the user and choose an LLM a_t according to a policy defined by $\pi_t : \mathcal{X} \rightarrow \Delta(\mathcal{A})$, where $\Delta(\mathcal{A})$ is the probability simplex over

set $\mathcal{A}$. Then the router observes a cost $c_t = \mathcal{C}(x_t, a_t)$ and reward $r_t = \mathcal{R}(x_t, a_t)$. Note that the interaction terminates after the budget is depleted. That means the router gets no rewards after the budget is exhausted. The goal of the router is to maximize the cumulative reward under the constrained resource budget. For reward and cost, we make the following realizability assumption.

ASSUMPTION 1. *There exists reward function $f : \mathcal{X} \times \mathcal{A} \rightarrow [0, F]$ and cost functions $g : \mathcal{X} \times \mathcal{A} \rightarrow [0, G]$ satisfy that $f(x, a) = \mathbb{E}[\mathcal{R}(x, a)]$ and $g(x, a) = \mathbb{E}[\mathcal{C}(x, a)]$, $\forall x \in \mathcal{X}, a \in \mathcal{A}$.*

Let $a_t^{\pi_t}$ be the action drawn by the policy $\pi_t(x_t)$ given context x_t in round t , i.e., $a_t^{\pi_t} \sim \pi_t(x_t)$. For the given sequence $\{x_t\}_t$, the router's objective is to design dynamic policies $\{\pi_t\}_t$ that maximize the cumulative rewards over horizon T under *the knapsack constraints*:

$$\max \sum_{t=1}^T f(x_t, a_t^{\pi_t}), \quad \text{s.t.} \quad \sum_{t=1}^T g(x_t, a_t^{\pi_t}) \leq B. \quad (1)$$

In fact, since the router does not know the reward and cost functions, and the distribution of queries $\mathcal{X}$ is unknown, solving the optimization problem (1) is highly challenging, especially under the knapsack constraint, where the accumulation of cost will affect the subsequent routing policy.

Regret. To evaluate the performance of a policy, we define regret through the following offline baseline problem with the assumption of full knowledge of rewards, costs, and context distribution. Define $b := B/T$, the baseline problem for (1) can be formulated as

$$\max_{\pi} \mathbb{E}_{a \sim \pi} [f(x, a)], \quad \text{s.t.} \quad \mathbb{E}_{a \sim \pi} [g(x, a)] \leq b, \quad (2)$$

with optimal policy π^* and optimal value v^* . The optimal value to problem (2) over a time horizon T , denoted as Tv^* , provides an upper bound on the optimal value of problem (1), which has already been discussed in Devanur et al. [6]. Therefore, the (pseudo-)regret can be defined as

$$\text{Regret}(T) = Tv^* - \sum_{t=1}^T \mathbb{E}_{a_t \sim \pi_t} [f(x_t, a_t)].$$

Online Regression Oracles. Since the reward and cost functions f and g are unknown to the router, and furthermore, the distribution of user queries is also unknown, it is necessary to employ a predictor that can learn the reward and cost online. We assume access to online regression oracles $\{\mathcal{R}\}_{f,g}$ for approximating the reward and cost functions during the online process. These oracles operate by minimizing the cumulative regret with respect to a given loss function over a sequence of predictions. Specifically, we employ the least squares regression approach formulated as $\ell = (y - \hat{y})^2$, where $\hat{y}$ denotes the prediction generated by the online regression oracle for y . With a new instance $z_t = (x_t, a_t) \in \mathcal{X} \times \mathcal{A}$, the oracles calculate the approximate functions $\hat{f}_t$ and $\hat{g}_t$ base on $\{(z_1, r_1, c_1), \dots, (z_{t-1}, r_{t-1}, c_{t-1})\}$. The online regression oracle is general and flexible, allowing for implementation using multiple models, such as neural networks or clustering methods. We make the following assumption for reward and cost regression oracles.

ASSUMPTION 2 (REGRESSION ORACLE). Let $\{x_t, a_t\}$ be the trajectory generated by a policy π . Let $\hat{f}_t$ and $\hat{g}_t$ calculated by regression oracle $\{\mathcal{R}\}_{f,g}$. With the definition of the squared regression $\epsilon_t^f(x_t, a_t) = (\hat{f}_t(x_t, a_t) - f(x_t, a_t))^2$ and $\epsilon_t^g(x_t, a_t) = (\hat{g}_t(x_t, a_t) - g(x_t, a_t))^2$, the following error bounds

$$\sum_{t=1}^T \epsilon_t^f(x_t, a_t) \leq U_f(p), \quad \sum_{t=1}^T \epsilon_t^g(x_t, a_t) \leq U_g(p),$$

holds with a probability $1 - p$ and define $U(p) = \max\{U_f(p), U_g(p)\}$ as the upper bound of the regression oracle errors.

REMARK 1. There exists online regression oracles that guarantee sublinear square error, i.e., $U(p) = o(T)$. For example, for finite hypothesis classes, there are regression oracles such that $U_f(p) = O(\log(|\mathcal{F}|/p))$ and $U_g(p) = O(\log(|\mathcal{G}|/p))$ [8]. We let $p = \frac{1}{T^2}$ in this paper.

4 Algorithm

Algorithm 1 Budget-Adaptive Router (BARouter)

- 1: **Input:** Time horizon T , Total budget B , LLM Set $\mathcal{A}$, Exploration factor η , Error bound U .
- 2: **Initialization:** Per-round budget $b = \frac{B}{T}$, $Q_1 = 0$ and $V = b\sqrt{TU}$, online regression oracles set $\{\mathcal{R}\}_{f,g}$.
- 3: **for** $t = 1, \dots, T$ **do**
- 4: **Parameters Estimation:** Given a query $x_t \sim \mathcal{X}$, estimate the reward functions $\hat{f}_t(x_t, a)$ and the cost function $\hat{g}_t(x_t, a)$ from regression oracles $\{\mathcal{R}\}_{f,g}$ for all $a \in \mathcal{A}$.
- 5: **Calculate Utility Function:**

$$w_a = \hat{f}_t(x_t, a) - \frac{Q_t}{V} \hat{g}_t(x_t, a), \quad \forall a \in \mathcal{A} \quad (3)$$
- 6: **Derive Exponential Policy:**

$$\pi_t(a) = \frac{\exp(\eta w_a)}{\sum_{i \in \mathcal{A}} \exp(\eta w_i)} \quad (4)$$
- 7: Select an LLM with $a_t \sim \pi_t$.
- 8: **Feedback and Oracles Update:** Generate the response y_t with LLM a_t , and observe the reward r_t and cost c_t . Feed the observation reward and cost to $\{\mathcal{R}\}_{f,g}$.
- 9: **Budget Pacing:** Update virtual queues as follows

$$Q_{t+1} = \max\{Q_t + c_t - b, 0\} \quad (5)$$
- 10: **end for**

We introduce **Budget-Adaptive Router (BARouter)** in Algorithm 1, a budget-adaptive routing framework with bandit-feedback. BARouter does not require any prior knowledge except the time horizon T and total budget B . Unlike the existing routing algorithms that control the cost consumption implicitly as discussed above, BARouter takes budget B as input directly, exhibiting stronger alignment with real-world application scenarios. Inspired by stochastic networking optimization [23], we design a “virtual queue” Q_t to track cumulative overused budget and dynamically regulate routing decisions. For an incoming prompt at round t , BARouter calculates the decision function using a budget-scaled virtual queue $\frac{Q_t}{V}$. It then derives an exponential policy with an exploration factor η and

samples an action from this policy. Once the router observes the feedback from the selected LLM. Subsequently, the online learning oracles and the virtual queue Q_t are updated based on the feedback, where Q_t incorporates the overused budget $c_t - b$. Specifically, BARouter has the following key components:

Budget Adaptive Multiplier. For an incoming query x_t , BARouter computes a unified utility function for every candidate LLM in (3). This approach resembles the primal-dual method, in which the primal component involves maximizing an approximated Lagrangian function: $L(x, a) = r(x, a) - \lambda c(x, a)$. The difference of BARouter lies in the fact that its Lagrange multiplier λ is dynamically estimated by Q_t/V in round t , which constitutes the key mechanism enabling more precise budget control. The design of virtual queue Q_t and budget-aware parameter V are explained as follows.

- The concept of virtual queues originates from queuing theory. The virtual queues are designed to track cumulative budget overruns through the update rule given in (5). Intuitively, Q_t reflects the magnitude of the user’s current budget overruns. Whenever the cost in a given round exceeds the average per-round budget allocation, Q increases and causes BARouter to adopt a more conservative strategy in subsequent rounds until Q returns to a lower level.
- The budget-aware parameter $V = b\sqrt{TU}$ functions similarly to an adaptive learning rate. Intuitively, a shorter time horizon T or a small budget B (a small average budget b) leads to a more conservative strategy. Conversely, a larger available budget or time horizon promotes a more optimistic strategy.

Due to the design of the budget adaptive multiplier, BARouter is capable of controlling costs more stably, ensuring that the budget is precisely depleted at the end of the query sequence.

Plug-in Predictor. The BARouter is a flexible to integrate any predictors with prior knowledge. Specifically, when an offline dataset is available, it can incorporate a pre-trained predictor trained on the dataset to achieve more accurate estimations of both reward and cost. Moreover, when confronted with queries from entirely new domains or a completely new LLM, the BARouter can also employ an online learning predictor to conduct appropriate exploration during interaction. This approach reduces prediction error while ensuring long-term response quality and budget control.

Exponential-weighting Principle to Balance Exploration and Exploitation (EXP3). BARouter adopts an EXP3-style design inspired by adversarial bandit algorithms. This design has several advantages: it is computationally efficient, as it does not require the construction of confidence sets as in optimistic exploration methods, and it is inherently robust to non-stationary or adversarial query distributions. By maintaining a probability distribution over candidate models and sampling according to the policy derived from (4), BARouter achieves a principled balance between exploration and exploitation. In contrast, optimistic or greedy approaches such as AUPD [12] rely on explicit optimistic estimations of both reward and cost to encourage exploration, which can be computationally and statistically challenging in the context of LLM routing.

5 Theoretical Results

In this section, we analyze the regret bounds of BARouter as in Theorem 5.1.

THEOREM 5.1. *Under Assumptions 1 and 2, BARouter achieves the following regret that*

$$\text{Regret}(T) = \tilde{O}(T^{\frac{3}{4}}U^{\frac{1}{4}}).$$

Inspired by Agrawal and Devanur [1], Guo and Liu [11, 12], we obtained the regret bounds in Theorem 5.1 as follows.

Regret Decomposition. The routing process can be decomposed into two phases: the model selection phase, when budget remains available, and the termination phase after budget exhaustion. We decouple the regrets of these two phases: *selection phase* regret stems from the performance gap between the selected model and the optimal model chosen by the optimal policy; *termination phase* regret arises from the opportunity cost of being unable to invoke any model. The two phases are demarcated by a stopping time τ , defined as the first round in which the budget is exceeded, i.e.,

$$\tau = \arg \min_{t \in [T]} \left\{ t \mid \sum_{s=1}^t g(x_s, a_s) \geq B \right\}.$$

Recall that π^* is the optimal solution to the baseline problem (2) and a_t^* is the action sampled from it, i.e., $a_t^* \sim \pi^*(x_t)$. We decompose the regret as follows

$$\begin{aligned} \text{Regret}(T) &= T v^* - \sum_{t=1}^{\tau} \mathbb{E}_{a_t \sim \pi_t} [f(x_t, a_t)] \\ &= \underbrace{v^* \mathbb{E}[T - \tau]}_{\text{termination phase regret}} + \underbrace{\sum_{t=1}^{\tau} [f(x_t, a_t^*) - \mathbb{E}_{a_t \sim x_t} [f(x_t, a_t)]]}_{\text{selection phase regret: Regret}(\tau)}. \end{aligned} \quad (6)$$

Regret Analysis in Selection Phase. Similar to Guo and Liu [11, 12], we analyze the regret via *Lyapunov drift analysis*. Due to page limitations, we present a proof sketch here, and more details can be found in Appendix B.1. Define the one step regret as $\text{Reg}(x_t, a) = f(x_t, a) - \mathbb{E}_{a_t \sim \pi_t} [f(x_t, a_t)]$, the Lyapunov drift as $\Delta_t = \frac{1}{2}Q_{t+1}^2 - \frac{1}{2}Q_t^2$ and let the filtration $\mathcal{H}_t = [x_t, \hat{f}_t, \hat{g}_t, Q_t]$. With the EXP3 policy construction method (4), we can establish an upper bound of the “one-step regret + Lyapunov drift” for any policy π :

$$\begin{aligned} &\mathbb{E} \left[\text{Reg}(x_t, a) + \frac{\Delta_t}{V} \mid \mathcal{H}_t \right] \\ &\leq \frac{G^2}{2V} + \frac{\ln |\mathcal{A}|}{\eta} + \frac{1}{\gamma} \left(1 + \frac{Q_t}{V} \right) + \frac{\gamma}{2} \mathbb{E}_{a \sim \pi} [\epsilon_t^f(x_t, a) + \frac{Q_t}{V} \epsilon_t^g(x_t, a) \mid \mathcal{H}_t] \\ &\quad + \frac{\gamma}{2} \mathbb{E}_{a_t \sim \pi_t} [\epsilon_t^f(x_t, a_t) + \frac{Q_t}{V} \epsilon_t^g(x_t, a_t) \mid \mathcal{H}_t], \end{aligned} \quad (7)$$

where γ is a parameter introduced to obtain the squared regression errors by AM-GM inequality. Note that the above inequality (7) holds for any policy π , including the optimal static policy π^* . Therefore, by setting $\pi = \pi^*$, we can obtain an anytime bound of the virtual queues $\{Q_t\}$ as Lemma 5.2, with $V = b\sqrt{TU}$, $\gamma = \sqrt{\frac{T}{U}}$, and $\eta = \sqrt{T}$.

LEMMA 5.2 (UPPER BOUND OF VIRTUAL QUEUES). *Under Assumptions 1 and 2, BARouter achieves that*

$$\mathbb{E}[Q_t] = \tilde{O}(T^{\frac{3}{4}}U^{\frac{1}{4}}).$$

Furthermore, the selection phase regret $\text{Regret}(\tau)$ can be derived by incorporating the upper bound on the virtual queues $\{Q_t\}$ in Lemma 5.2 into inequality (7) and resulting Lemma 5.3.

LEMMA 5.3 (SELECTION PHASE REGRET). *Under Assumption 1 and 2, BARouter achieves that*

$$\mathbb{E}[\text{Regret}(\tau)] = \tilde{O}(T^{\frac{3}{4}}U^{\frac{1}{4}}).$$

Regret Analysis in Termination Phase. The key to analyzing termination phase regret lies in determining the skipping rounds $(T - \tau)$, which is bounded by the virtual queues, as discussed in Appendix B.2.

LEMMA 5.4 (TERMINATION PHASE REGRET). *Under Assumptions 1 and 2, the expected remaining round under BARouter satisfies that*

$$\mathbb{E}[T - \tau] = \tilde{O}(T^{\frac{3}{4}}U^{\frac{1}{4}}/b),$$

and the termination phase regret achieves that

$$v^* \mathbb{E}[T - \tau] = \tilde{O}(\frac{v^*}{b} T^{\frac{3}{4}}U^{\frac{1}{4}}).$$

Proving Theorem 5.1. As shown in Equation (6), the overall regret bound in Theorem 5.1 is derived by combining the regret from both the selection and termination phases:

$$\begin{aligned} \text{Regret}(T) &\leq v^* \mathbb{E}[T - \tau] + \mathbb{E} \left[\sum_{t=1}^{\tau} f(x_t, a_t^*) - f(x_t, a_t) \right] \\ &= \tilde{O}(\frac{v^*}{b} T^{\frac{3}{4}}U^{\frac{1}{4}} + T^{\frac{3}{4}}U^{\frac{1}{4}}), \end{aligned}$$

which completes the proof, and more details can be found in the Appendix B.3.

6 Experiments

In this section, we empirically address the following questions:

- How does BARouter perform relative to state-of-the-art baselines across benchmark tasks?
- How does BARouter maintain robustness under dynamic query distribution?

We address these questions through benchmark results evaluations and robustness analysis under query distribution shifts.

Dataset and Benchmarks. The experiments were employed on two publicly available benchmarks for LLM routing, **RouterBench** [14] and **SPROUT** [28]. RouterBench comprises 35,712 prompts with responses from 11 LLMs, while SPROUT contains 44,241 prompts with responses from 13 LLMs. Regarding the global budget, we define the unit of budget B as US dollars (\$).

Query Embedding and Plug-in Predictors. To handle queries, a common first step is to map the query text into a high-dimensional vector space using a text embedding model, and then use this representation as input to the predictors. In our experiments, we employ the bert-base-uncased¹ as the text embedding model, which encodes

¹<https://huggingface.co/google-bert/bert-base-uncased>

Table 2: Averaged reward across different scenarios on different routers. The results are reported over 5 trials with different random seeds. We bold the best performance in each row. Since the Primal-Dual Router does not rely on predictors, we merge the cells in this column.

Benchmark	Budget (\$)	Predictor	Fixed Cost-sensitivity Router	Primal-Dual Router	Ratio Router	Conservative Router	BARouter (Ours)
RouterBench	$B = 2$	KNN	0.427 ± 0.002	0.557 ± 0.001	0.431 ± 0.001	0.546 ± 0.002	0.590 ± 0.004
		K-Means	0.517 ± 0.001		0.467 ± 0.001	0.508 ± 0.001	0.595 ± 0.002
		Gradient Boosting	0.454 ± 0.001		0.347 ± 0.002	0.472 ± 0.001	0.494 ± 0.001
	$B = 5$	KNN	0.651 ± 0.001	0.660 ± 0.005	0.431 ± 0.001	0.653 ± 0.003	0.675 ± 0.003
		K-Means	0.630 ± 0.000		0.467 ± 0.001	0.617 ± 0.001	0.686 ± 0.002
		Gradient Boosting	0.624 ± 0.001		0.618 ± 0.001	0.552 ± 0.001	0.626 ± 0.001
	$B = 10$	KNN	0.484 ± 0.008	0.684 ± 0.006	0.431 ± 0.000	0.672 ± 0.001	0.716 ± 0.003
		K-Means	0.679 ± 0.006		0.467 ± 0.000	0.636 ± 0.001	0.718 ± 0.004
		Gradient Boosting	0.640 ± 0.001		0.618 ± 0.001	0.623 ± 0.001	0.638 ± 0.001
SPROUT	$B = 0.5$	KNN	0.637 ± 0.001	0.540 ± 0.005	0.637 ± 0.000	0.656 ± 0.004	0.659 ± 0.007
		K-Means	0.614 ± 0.006		0.606 ± 0.001	0.631 ± 0.001	0.651 ± 0.001
		Gradient Boosting	0.648 ± 0.001		0.394 ± 0.001	0.568 ± 0.000	0.648 ± 0.002
		Roberta (offline)	0.680 ± 0.000		0.633 ± 0.001	0.669 ± 0.001	0.668 ± 0.002
	$B = 0.9$	KNN	0.696 ± 0.003	0.603 ± 0.004	0.637 ± 0.001	0.690 ± 0.002	0.722 ± 0.006
		K-Means	0.708 ± 0.000		0.606 ± 0.002	0.704 ± 0.000	0.715 ± 0.006
		Gradient Boosting	0.647 ± 0.007		0.705 ± 0.005	0.602 ± 0.002	0.710 ± 0.001
		Roberta (offline)	0.711 ± 0.000		0.633 ± 0.001	0.701 ± 0.001	0.703 ± 0.001
	$B = 1.6$	KNN	0.709 ± 0.003	0.716 ± 0.004	0.637 ± 0.002	0.756 ± 0.004	0.777 ± 0.004
		K-Means	0.594 ± 0.005		0.606 ± 0.001	0.722 ± 0.002	0.786 ± 0.003
		Gradient Boosting	0.737 ± 0.004		0.720 ± 0.001	0.664 ± 0.003	0.798 ± 0.001
		Roberta (offline)	0.754 ± 0.002		0.633 ± 0.001	0.744 ± 0.005	0.791 ± 0.003

query text into a 768-dimensional vector. As for predictors, virtually all routing algorithms require predictors to estimate the reward and cost of a given query x on each available LLM a , denoted as $\hat{f}(x, a)$ and $\hat{g}(x, a)$, respectively. We consider the following predictors in our experimental setup:

- **K-nearest-neighbors(KNN)** The KNN regressor is a commonly employed estimator in LLM routing tasks [14, 18, 28], characterized by its ability to operate effectively without requiring extensive training set.
- **K-Means.** We performed K-Means clustering based on the embeddings. For each cluster, we estimate the reward or cost as its average value.
- **Gradient Boosting.** We utilize gradient-boosted tree regression to do estimation. We concatenate the embeddings of user prompts and available LLMs as input to the regressor to fit reward and cost.
- **Roberta.** Somerstep et al. [28] has open-sourced their predictor trained on the SPROUT dataset on Hugging Face, which is based on a fine-tuned RoBERTa-base architecture.

To simulate the online scenario, KNN, K-Means, and Gradient Boosting are adapted to acquire bandit feedback and can be updated during routing. For these online predictors, we established 200 queries as the initial dataset. That is, the predictor possesses a dataset comprising responses from all available LLMs to 200 questions. As for Roberta, it was pre-trained on the SPROUT training set (30968 samples) and is not updated during the routing process. Detailed settings for the predictors can be found in the Appendix C.1.

Baselines. In the experiment, we considered multiple baseline algorithms, which can be classified into the following categories with detailed description in Appendix C.2:

- **Fixed Cost-sensitivity Router.** Hu et al. [14], Jitkrittum et al. [18], Somerstep et al. [28] adjust the sensitivity to the cost through tuning hyperparameters. Given a query x , they compute a score for each available LLM a and then select the LLM with the highest score. For carrot [28], $s_a = (1 - \mu)\hat{f}(x, a) - \mu\hat{g}(x, a)$, where $\hat{f}(x, a)$ and $\hat{g}(x, a)$ represent the predicted reward and cost, respectively, μ is the **fixed** cost-sensitivity parameter. Although the other two methods have different computational forms, they are essentially convertible into each other.
- **Primal-Dual Router.** OmniRouter [21] formulates the LLM routing problem as an integer programming problem with N queries and M LLMs. Notably, this approach constitutes an offline optimization problem and cannot adapt to online scenarios. To address this challenge, we first categorize queries. For each category of queries, we compute the probability of assigning a query to each LLM and apply it as a policy in online routing.
- **Conservative Router.** Zhao et al. [34] ranks the candidate LLMs based on preference feedback and subsequently select the highest-ranking LLM within the user’s budget. Inspired by this approach, Conservative Router selects the LLM with the highest response quality, subject to the constraint that the cost does not exceed the current budget per query.
- **Ratio Router.** Ratio router chooses the LLM with the highest predicted reward-cost ratio.

6.1 Benchmark Results

We evaluate BARouter in comparison to the aforementioned baselines on both RouterBench and SPROUT.

Experiments Setup. In the RouterBench dataset, we set the time horizon to $T = 20000$ and defined three distinct global budget $B = \{2, 5, 10\}$. For the SPROUT dataset, the time horizon was set to $T = 5309$, with global budget $B = \{0.5, 0.9, 1.6\}$. Regarding the hyperparameter settings, for the Fixed Cost-sensitivity Router, based on CARROT[28], we adjust the cost weight μ . For different budgets $B = \{2, 5, 10\}$ in RouterBench, we set $\mu = \{0.98, 0.8, 0.25\}$. For $B = \{0.5, 0.9, 1.6\}$ in SPROUT, we configure $\mu = \{0.95, 0.9, 0.4\}$. In contrast, BARouter requires no hyperparameter tuning to adapt to various budgets. For η and U , which denote the exploration factor and the predictor error bound, respectively, we set $\eta = 20$ and $U = 1$ across all scenarios.

Experiments Results and Discussion. The experiment results are shown in Table 2. As can be observed from the table, BARouter outperforms the baseline methods under various budget settings across both datasets. BARouter uses a single hyperparameter setting $\eta = 20, U = 1$ across all scenarios, whereas the Fixed Cost-sensitivity Router requires meticulous hyperparameter tuning. The Fixed Cost-sensitivity Router shows suboptimal performance with certain predictors under the same hyperparameters. For instance, while the hyperparameter $\mu = 0.25$ is quite appropriate for the Fixed Cost-sensitivity Router with K-Means under a budget of $B = 10$, it leads to poor results when Online KNN is applied, induces an overly aggressive strategy, resulting in premature budget exhaustion, which makes the performance even worse than that at $B = 5$. This verifies that hyperparameter selection for Fixed Cost-sensitivity Router is highly dependent on factors such as the predictor, making it difficult to choose optimally in practical deployment. It also demonstrates BARouter’s strong adaptability to different budget scenarios, as it precisely controls the global budget under varying budget constraints.

6.2 Robustness under Dynamic Distribution

In practical deployment, the data distribution $\mathcal{X}$ may be dynamic and unobservable. This is a challenge for traditional offline or static methods [14, 18, 28], which require a precise configuration of the user’s cost sensitivity (λ or μ) to control the global reward-cost trade-off. However, BARouter dynamically optimizes the reward-cost trade-off in real-time without prior knowledge of data distributions through its budget-adaptation mechanism. To demonstrate BARouter’s robustness under dynamic distribution, we introduce the following experiments.

Experiments Setup. We utilize two models from RouterBench with contrasting capability-cost profiles: a strong LLM (gpt-4-1106-preview) and a weak LLM (meta/llama-2-70b-chat). The queries in RouterBench are divided into an easy subset $\mathcal{X}_{\text{easy}}$ and a hard subset $\mathcal{X}_{\text{hard}}$ based on the weak LLM’s correctness. We consider two different dynamic distribution settings:

- The time horizon $T = 25000$ are divided into two stages with budget $B = 20$. For the first stage $t \in [0, 12500]$, prompts are sampled from $\mathcal{X}_{\text{easy}}$ and $\mathcal{X}_{\text{hard}}$ in an 8 : 2 ratio; while for the second stage $t \in [12500, 25000]$, in a 2 : 8 ratio.

- The prompt distribution from $\mathcal{X}_{\text{easy}} : \mathcal{X}_{\text{hard}} = 2 : 8$ linearly varies to $\mathcal{X}_{\text{easy}} : \mathcal{X}_{\text{hard}} = 8 : 2$ during time horizon $T = 25000$ with budget $B = 20$.

We use the KNN predictor in these experiments and compare BARouter with baselines. In this experiment, we apply $s_a = \hat{r}(x, a) - \lambda \hat{c}(x, a)$ for the Fixed Cost-sensitivity Router. To ensure a competitive baseline, we meticulously selected parameters for the fixed cost-sensitivity routers, specifically $\lambda = \{120, 150\}$. These values represent near-optimal choices under two idealized scenarios: full knowledge of the initial query distribution, and prior awareness of global distribution shifts. The detailed parameter selection procedure is provided in Appendix C.3.

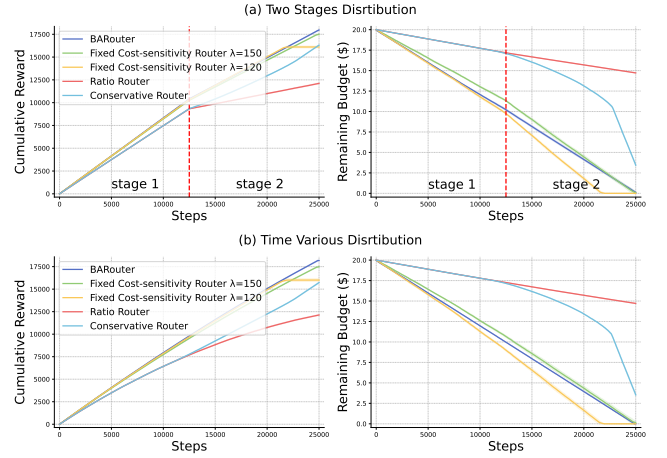


Figure 3: Online router reward over different dynamic distributions.

Experiments Results and Discussion. The experiment results are shown in Figure 3. As evidenced by the results, BARouter demonstrates superior performance over baseline algorithms under dynamic distribution shifts. Although the fixed cost-sensitivity routers with meticulously tuned parameters achieve cumulative rewards comparable to our method, such fine-grained parameter calibration in Appendix C.3 proves impractical to attain in realistic online deployment scenarios.

7 Conclusion

In this work, we introduce BARouter, a budget-adaptive online LLM routing framework that optimizes response quality under various budget scenarios without requiring parameter tuning. Theoretically, we provide a $\tilde{O}(T^{\frac{3}{4}}U^{\frac{1}{4}})$ regret for BARouter. Through experiments, we demonstrate that BARouter outperforms existing LLM routing methods, and also validate its adaptability and strong budget control capabilities.

Acknowledgment

This work was supported by the National Natural Science Foundation of China under Grant 62302305.

References

- [1] Shipra Agrawal and Nikhil Devanur. 2016. Linear Contextual Bandits with Knapsacks. In *Advances Neural Information Processing Systems (NeurIPS)*.
- [2] Anthropic. 2024. The Claude 3 Model Family: Opus, Sonnet, Haiku. *Claude-3 Model Card* (2024). <https://api.semanticscholar.org/CorpusID:268232499>
- [3] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. 2023. Qwen technical report. *arXiv preprint arXiv:2309.16609* (2023).
- [4] Shuhao Chen, Weisen Jiang, Baijiong Lin, James Kwok, and Yu Zhang. 2024. Routerdc: Query-based router by dual contrastive learning for assembling large language models. In *Advances Neural Information Processing Systems (NeurIPS)*.
- [5] Xiangxiang Dai, Jin Li, Xutong Liu, Anqi Yu, and John Lui. 2024. Cost-effective online multi-llm selection with versatile reward models. *arXiv preprint arXiv:2405.16587* (2024).
- [6] Nikhil R Devanur, Kamal Jain, Balasubramanian Sivan, and Christopher A Wilkens. 2019. Near optimal online algorithms and fast approximation algorithms for resource allocation problems. *J. ACM* 66, 1 (2019), 1–41.
- [7] Dujian Ding, Ankur Mallick, Shaokun Zhang, Chi Wang, Daniel Madrigal, Mirian Del Carmen Hipolito Garcia, Menglin Xia, Laks V. S. Lakshmanan, Qingyun Wu, and Victor Rühle. 2025. BEST-Route: Adaptive LLM Routing with Test-Time Optimal Compute. In *Int. Conf. Machine Learning (ICML)*.
- [8] Dylan J Foster and Alexander Rakhlin. 2023. Foundations of reinforcement learning and interactive decision making. *arXiv preprint arXiv:2312.16730* (2023).
- [9] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).
- [10] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).
- [11] Hengquan Guo and Xin Liu. 2024. Stochastic Constrained Contextual Bandits via Lyapunov Optimization Based Estimation to Decision Framework. In *The Thirty Seventh Annual Conference on Learning Theory*. PMLR, 2204–2231.
- [12] Hengquan Guo and Xin Liu. 2025. On Stochastic Contextual Bandits with Knapsacks in Small Budget Regime. In *Int. Conf. on Learning Representations (ICLR)*.
- [13] Surya Narayanan Hari and Matt Thomson. 2023. Tryage: Real-time, intelligent routing of user prompts to large language models. *arXiv preprint arXiv:2308.11601* (2023).
- [14] Qitian Jason Hu, Jacob Bieker, Xiuyu Li, Nan Jiang, Benjamin Keigwin, Gaurav Ranganath, Kurt Keutzer, and Shriyash Kaustubh Upadhyay. 2024. RouterBench: A Benchmark for Multi-LLM Routing System. In *Agentic Markets Workshop at ICML 2024*.
- [15] Xiaoyan Hu, Ho-fung Leung, and Farzan Farnia. 2025. An Online Learning Approach to Prompt-based Selection of Generative Models and LLMs. In *Int. Conf. Machine Learning (ICML)*.
- [16] Xiaoyan Hu, Ho-fung Leung, and Farzan Farnia. 2024. An online learning approach to prompt-based selection of generative models. *arXiv preprint arXiv:2410.13287* (2024).
- [17] Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, et al. 2024. Openai o1 system card. *arXiv preprint arXiv:2412.16720* (2024).
- [18] Wittawat Jitkrittum, Harikrishna Narasimhan, Ankit Singh Rawat, Jeevish Juneja, Congchao Wang, Zifeng Wang, Alec Go, Chen-Yu Lee, Pradeep Shenoy, Rina Panigrahy, et al. 2025. Universal model routing for efficient llm inference. *arXiv preprint arXiv:2502.08773* (2025).
- [19] Dongjun Lee, Juyong Lee, Kyuyoung Kim, Jihoon Tack, Jinwoo Shin, Yee Whye Teh, and Kimin Lee. 2025. Learning to contextualize web pages for enhanced decision making by LLM agents. *arXiv preprint arXiv:2503.10689* (2025).
- [20] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. In *Advances Neural Information Processing Systems (NeurIPS)*.
- [21] Kai Mei, Wujiang Xu, Shuhang Lin, and Yongfeng Zhang. 2025. OmniRouter: Budget and Performance Controllable Multi-LLM Routing. *arXiv preprint arXiv:2502.20576* (2025).
- [22] Alireza Mohammadshahi, Arshad Rafiq Shaikh, and Majid Yazdani. 2024. Routoo: Learning to route to large language models effectively. *arXiv preprint arXiv:2401.13979* (2024).
- [23] M. J. Neely. 2022. *Stochastic network optimization with application to communication and queueing systems*. Springer Nature.
- [24] Quang H Nguyen, Thinh Dao, Duy C Hoang, Juliette Decugis, Saurav Manchanda, Nitesh V Chawla, and Khoa D Doan. 2024. MetaLLM: A High-performant and Cost-efficient Dynamic Framework for Wrapping LLMs. *arXiv preprint arXiv:2407.10834* (2024).
- [25] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M Waleed Kadous, and Ion Stoica. 2025. RouteLLM: Learning to Route LLMs from Preference Data. In *Int. Conf. on Learning Representations (ICLR)*.
- [26] Zhihong Pan, Kai Zhang, Yuze Zhao, and Yupeng Han. 2025. Route to Reason: Adaptive Routing for LLM and Reasoning Strategy Selection. *arXiv preprint arXiv:2505.19435* (2025).
- [27] Tal Shnitzer, Anthony Ou, Mirian Silva, Kate Soule, Yuekai Sun, Justin Solomon, Neil Thompson, and Mikhail Yurochkin. 2024. Large Language Model Routing with Benchmark Datasets. In *First Conference on Language Modeling*.
- [28] Seamus Somerstrep, Felipe Maia Polo, Allysson Flavio Melo de Oliveira, Pratyush Mangal, Mirian Silva, Onkar Bhardwaj, Mikhail Yurochkin, and Subha Maity. 2025. CARROT: A Cost Aware Rate Optimal Router. In *ICLR 2025 Workshop on Foundation Models in the Wild*.
- [29] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805* (2023).
- [30] Asterios Tsourvas, Wei Sun, and Georgia Perakis. 2025. Causal LLM Routing: End-to-End Regret Minimization from Observational Data. *arXiv preprint arXiv:2505.16037* (2025).
- [31] Xidong Wang, Nuo Chen, Junyin Chen, Yidong Wang, Guorui Zhen, Chunxian Zhang, Xiangbo Wu, Yan Hu, Anningzhe Gao, Xiang Wan, et al. 2024. Apollo: A lightweight multilingual medical LLM towards democratizing medical AI to 6B people. *arXiv preprint arXiv:2403.03640* (2024).
- [32] Yu Xia, Fang Kong, Tong Yu, Liya Guo, Ryan A Rossi, Sungchul Kim, and Shuai Li. 2024. Which llm to play? convergence-aware online model selection with time-increasing bandits. In *Proc. Int. Conf. World Wide Web (WWW)*.
- [33] Ke Yang, Yao Liu, Sapana Chaudhary, Rasool Fakoor, Pratik Chaudhari, George Karypis, and Huzefa Rangwala. 2024. Agentoccam: A simple yet strong baseline for llm-based web agents. *arXiv preprint arXiv:2410.13825* (2024).
- [34] Zesen Zhao, Shuowei Jin, and Z Morley Mao. 2024. Eagle: Efficient training-free router for multi-llm inference. *arXiv preprint arXiv:2409.15518* (2024).

A Limitations and Feature Works

While the proposed BARouter establishes a viable paradigm for online LLM routing with effective budget control and demonstrates empirically strong performance, the approach nevertheless exhibits certain limitations. A notable limitation concerns predictor accuracy. BARouter's performance is highly dependent on the precision of its predictors; however, existing prediction methods, including K-means, KNN, and neural networks, often struggle to deliver highly accurate results. Therefore, identifying effective feature encoding schemes, model architectures, and training methodologies to develop more reliable predictors remains an open research challenge. This problem is closely tied to fundamental issues in pattern recognition and efficient embedding learning.

B Proof

B.1 Selection Phase Regret Analysis

Establish “Regret + Lyapunov drift” bound. In the first step of analyzing the selection phase regret, we should establish “Regret + Lyapunov drift” bound. We introduce the following lemma to bridge the EXP3 policy and other policies.

LEMMA B.1 (PERFORMANCE DIFFERENT). *Suppose that $L : \mathcal{A} \rightarrow \mathbb{R}$ is a performance function, $\pi(a) = \frac{\exp(\eta L(a))}{Z}$, where η is a parameter and $Z = \sum_{a \in \mathcal{A}} \exp(\eta L(a))$ is the normalize factor. The performance difference between a certain action a' and an EXP3 policy is*

$$L(a') - \mathbb{E}_{a \sim \pi} [L(a)] \leq \frac{\ln |\mathcal{A}|}{\eta}.$$

PROOF. Since $\pi(a) = \frac{\exp(\eta L(a))}{Z}$, we have $\eta L(a) = \ln \pi(a) + \ln Z$, then

$$\begin{aligned} \eta \mathbb{E}_{a \sim \pi} (L(a)) &= \mathbb{E}_{a \sim \pi} (\eta L(a)) \\ &= \sum_{a \in \mathcal{A}} \pi(a) [\ln \pi(a) + \ln Z] \\ &= \ln Z + \sum_{a \in \mathcal{A}} \pi(a) \ln \pi(a) \\ &= \ln Z - H(\pi), \end{aligned}$$

where $H(\pi) = -\sum_{a \in \mathcal{A}} \pi(a) \ln \pi(a)$ is the negative entropy and there is an upper bound $H(\pi) \leq \ln |\mathcal{A}|$ for it. Hence, we have

$$L(a') - \mathbb{E}_{a \sim \pi} [L(a)] = \frac{\ln \pi(a') + \ln Z - \ln Z + H(\pi)}{\eta} \leq \frac{\ln |\mathcal{A}|}{\eta}.$$

□

With Lemma B.1, let the performance function $L(a) = \hat{f}_t(x_t, a) - \frac{Q_t}{V} \hat{g}_t(x_t, a)$, we have

$$\hat{f}_t(x_t, a) - \frac{Q_t}{V} \hat{g}_t(x_t, a) \leq \mathbb{E}_{a_t \sim \pi_t} [\hat{f}_t(x_t, a_t) - \frac{Q_t}{V} \hat{g}_t(x_t, a_t)] + \frac{\ln |\mathcal{A}|}{\eta}.$$

Since $\text{Reg}(x_t, a) = f(x_t, a) - \mathbb{E}_{a_t \sim \pi_t} [f(x_t, a_t)]$, adding the one step regret term into above inequality obtains

$$\begin{aligned} \text{Reg}(x_t, a) &\leq [f(x_t, a) - \hat{f}_t(x_t, a)] + \mathbb{E}_{a_t \sim \pi_t} [\hat{f}_t(x_t, a_t) - f(x_t, a_t)] \\ &\quad + \frac{Q_t}{V} (\hat{g}_t(x_t, a) - \mathbb{E}_{a_t \sim \pi_t} [\hat{g}_t(x_t, a_t)]) + \frac{\ln |\mathcal{A}|}{\eta}. \end{aligned} \quad (8)$$

According to the virtual queue update

$$Q_{t+1} = \max\{Q_t + g_t(x_t, a_t) - b, 0\},$$

the following inequality holds for the Lyapunov drift that

$$\begin{aligned} &\mathbb{E}_{a_t \sim \pi_t} [\Delta_t \mid \mathcal{H}_t] \\ &:= \frac{1}{2} \mathbb{E}_{a_t \sim \pi_t} [Q_{t+1}^2 \mid \mathcal{H}_t] - \frac{1}{2} \mathbb{E}_{a_t \sim \pi_t} [Q_t^2 \mid \mathcal{H}_t] \\ &\leq \frac{1}{2} \mathbb{E}_{a_t \sim \pi_t} [(2Q_t + g_t(x_t, a_t) - b)(g_t(x_t, a_t) - b) \mid \mathcal{H}_t] \\ &= \mathbb{E}_{a_t \sim \pi_t} \left[Q_t (g_t(x_t, a_t) - b) + \frac{1}{2} (g_t(x_t, a_t) - b)^2 \mid \mathcal{H}_t \right]. \end{aligned}$$

Combine this inequality with (8), we can establish the following key bound on the “Regret + Lyapunov drift”:

$$\begin{aligned} &\mathbb{E} \left[\text{Reg}(x_t, a) + \frac{\Delta_t}{V} \mid \mathcal{H}_t \right] \\ &\leq \mathbb{E}_{a \sim \pi} [f(x_t, a) - \hat{f}_t(x_t, a) \mid \mathcal{H}_t] + \mathbb{E}_{a_t \sim \pi_t} [\hat{f}_t(x_t, a_t) - f(x_t, a_t) \mid \mathcal{H}_t] \\ &\quad + \frac{Q_t}{V} \mathbb{E}_{a_t \sim \pi_t} [g_t(x_t, a_t) - \hat{g}_t(x_t, a_t) \mid \mathcal{H}_t] \\ &\quad + \frac{Q_t}{V} \mathbb{E}_{a \sim \pi} [\hat{g}_t(x_t, a) - g_t(x_t, a) \mid \mathcal{H}_t] \\ &\quad + \frac{Q_t}{V} \mathbb{E}_{a \sim \pi} [g_t(x_t, a) - b \mid \mathcal{H}_t] + \frac{1}{2V} (\mathbb{E}_{a_t \sim \pi_t} [g_t(x_t, a_t) \mid \mathcal{H}_t] - b)^2 \\ &\quad + \frac{\ln |\mathcal{A}|}{\eta} \\ &\leq \frac{Y}{2} \mathbb{E}_{a_t \sim \pi_t} [\epsilon_t^f(x_t, a_t) + \frac{Q_t}{V} \epsilon_t^g(x_t, a_t) \mid \mathcal{H}_t] \\ &\quad + \frac{Y}{2} \mathbb{E}_{a \sim \pi} [\epsilon_t^f(x_t, a) + \frac{Q_t}{V} \epsilon_t^g(x_t, a) \mid \mathcal{H}_t] \\ &\quad + \frac{Q_t}{V} \mathbb{E}_{a \sim \pi} [g_t(x_t, a) - b \mid \mathcal{H}_t] + \frac{1}{2V} (\mathbb{E}_{a_t \sim \pi_t} [g_t(x_t, a_t) \mid \mathcal{H}_t] - b)^2 \\ &\quad + \frac{\ln |\mathcal{A}|}{\eta} + \frac{1}{Y} (1 + \frac{Q_t}{V}), \end{aligned}$$

where the second inequality holds because of the AM-GM inequality. Let $\pi = \pi^*$, then above regret have

$$\begin{aligned} &\mathbb{E} \left[\text{Reg}(x_t, a) + \frac{\Delta_t}{V} \mid \mathcal{H}_t \right] \\ &\leq \frac{G^2}{2V} + \frac{\ln |\mathcal{A}|}{\eta} + \frac{1}{Y} (1 + \frac{Q_t}{V}) \\ &\quad + \frac{Y}{2} \mathbb{E}_{a_t \sim \pi_t} [\epsilon_t^f(x_t, a_t) + \frac{Q_t}{V} \epsilon_t^g(x_t, a_t) \mid \mathcal{H}_t] \\ &\quad + \frac{Y}{2} \mathbb{E}_{a \sim \pi} [\epsilon_t^f(x_t, a) + \frac{Q_t}{V} \epsilon_t^g(x_t, a) \mid \mathcal{H}_t], \end{aligned} \quad (9)$$

since the optimal solution satisfies the constraints and Assumption 1.

Upper Bound of Virtual Queues. From the inequality 9, we can obtain the upper bound of virtual queues presented in lemma 5.2 as follows.

$$\begin{aligned} \mathbb{E}[\frac{1}{2} Q_{t+1}^2] &= \sum_{s=1}^t \mathbb{E}[\Delta_s \mid \mathcal{H}_s] \\ &\leq V \sum_{s=1}^t (\mathbb{E}_{a_s \sim \pi_s} [f(x_s, a_s) \mid \mathcal{H}_s] - \mathbb{E}_{a \sim \pi^*} [f(x_s, a) \mid \mathcal{H}_s]) \\ &\quad + \sum_{s=1}^t (\frac{G^2}{2} + \frac{V \ln |\mathcal{A}|}{\eta} + \frac{1}{Y} (V + Q_s)) \end{aligned}$$

$$\begin{aligned}
& + \frac{\gamma}{2} \sum_{s=1}^t \mathbb{E}_{a_s \sim \pi_s} \left[V \epsilon_s^f(x_s, a_s) + Q_s \epsilon_s^g(x_s, a_s) \mid \mathcal{H}_s \right] \\
& + \frac{\gamma}{2} \sum_{s=1}^t \mathbb{E}_{a \sim \pi} \left[V \epsilon_s^f(x_s, a) + Q_s \epsilon_s^g(x_s, a) \mid \mathcal{H}_s \right] \\
& \leq 2FVt + \frac{tG^2}{2} + \frac{tV \ln |\mathcal{A}|}{\eta} + \frac{tV}{\gamma} + \frac{1}{\gamma} \sum_{s=1}^t Q_s \\
& + \frac{\gamma}{2} \sum_{s=1}^t \mathbb{E}_{a_s \sim \pi_s} \left[V \epsilon_s^f(x_s, a_s) + Q_s \epsilon_s^g(x_s, a_s) \mid \mathcal{H}_s \right] \\
& + \frac{\gamma}{2} \sum_{s=1}^t \mathbb{E}_{a \sim \pi} \left[V \epsilon_s^f(x_s, a) + Q_s \epsilon_s^g(x_s, a) \mid \mathcal{H}_s \right] \\
& \leq 2FVt + \frac{tG^2}{2} + \frac{tV \ln |\mathcal{A}|}{\eta} + \frac{tV}{\gamma} + \frac{t^2G}{\gamma} \\
& + \frac{\gamma}{2} \sum_{s=1}^t \mathbb{E}_{a_s \sim \pi_s} \left[V \epsilon_s^f(x_s, a_s) + tG \epsilon_s^g(x_s, a_s) \mid \mathcal{H}_s \right] \\
& + \frac{\gamma}{2} \sum_{s=1}^t \mathbb{E}_{a \sim \pi} \left[V \epsilon_s^f(x_s, a) + tG \epsilon_s^g(x_s, a) \mid \mathcal{H}_s \right] \\
& \leq 2FVT + \frac{TG^2}{2} + \frac{TV \ln |\mathcal{A}|}{\eta} + \frac{TV}{\gamma} + \frac{T^2G}{\gamma} + \gamma VU_r + \gamma TGU_c.
\end{aligned}$$

Take $V = b\sqrt{TU}$, $\gamma = \sqrt{T/U}$ and $\eta = \sqrt{T}$ into above inequality, we have

$$\mathbb{E}[Q_{t+1}^2] \leq \hat{B}_Q = (4Fb + 4G)T^{\frac{2}{3}}U^{\frac{1}{2}} + 2bTU + b(\ln |\mathcal{A}| + G^2)T,$$

which provides an anytime bound that $\mathbb{E}[Q_t] = \tilde{O}(T^{\frac{3}{4}}U^{\frac{1}{4}})$, $\forall t \in [T]$ resulting of Lemma 5.2. Specifically, we prove that

$$\mathbb{E}[Q_{t+1}] \leq C_1 T^{\frac{3}{4}}U^{\frac{1}{4}},$$

where $C_1 = 2\sqrt{Fb} + G$.

Compute Regret of Selection Phase. With the upper bound of the virtual queues, we can obtain the regret bound of the selection phase as follows,

$$\begin{aligned}
& \text{Regret}(\tau) \\
& = \sum_{t=1}^{\tau} \mathbb{E}_{a \sim \pi^*} [f(x_t, a)] - \mathbb{E}_{a_t \sim \pi_t} [f_t(x_t, a_t)] \\
& \leq \sum_{t=1}^{\tau} \frac{G^2}{2V} + \frac{\ln |\mathcal{A}|}{\eta} + \frac{1}{\gamma} \left(1 + \frac{Q_t}{V}\right) - \mathbb{E} \left[\frac{\Delta_t}{V} \mid \mathcal{H}_t \right] \\
& + \frac{\gamma}{2} \sum_{s=1}^t \mathbb{E}_{a_s \sim \pi_s} \left[\epsilon_s^f(x_s, a_s) + \frac{Q_t}{V} \epsilon_s^g(x_s, a_s) \mid \mathcal{H}_s \right] \\
& + \frac{\gamma}{2} \sum_{s=1}^t \mathbb{E}_{a \sim \pi} \left[\epsilon_s^f(x_s, a) + \frac{Q_t}{V} \epsilon_s^g(x_s, a) \mid \mathcal{H}_s \right] \\
& \leq \frac{G^2T}{2V} + \frac{T \ln |\mathcal{A}|}{\eta} + \frac{T}{\gamma} + \frac{Q_{\max}T}{V\gamma} + \gamma U_f + \gamma \frac{Q_{\max}}{V} U_g - \sum_{t=1}^{\tau} \frac{\mathbb{E}[\Delta_t]}{V} \\
& \leq \frac{G^2T}{2V} + \frac{T \ln |\mathcal{A}|}{\eta} + \frac{T}{\gamma} + \gamma U + \left(\frac{T}{V\gamma} + \frac{\gamma U}{V} \right) Q_{\max},
\end{aligned}$$

which provides the regret of the selection phase $\text{Regret}(\tau) = \tilde{O}(T^{\frac{3}{4}}U^{\frac{1}{4}})$ as Lemma 5.3. The last inequality holds since $\sum_{t=1}^{\tau} \frac{\mathbb{E}[\Delta_t]}{V} = \frac{1}{2}Q_{\tau+1}^2 - \frac{1}{2}Q_1^2 = \frac{1}{2}Q_{\tau+1}^2 \geq 0$ with $Q_1 = 0$.

B.2 Termination Phase Regret Analysis

The key to proving the termination phase regret line in determining the skipping rounds $(T - \tau)$. According to the virtual queue update in (5.2), we have

$$Q_{\tau+1} + \tau b \geq \sum_{t=1}^{\tau} g(x_t, a_t),$$

in conjunction with the definition of stopping time, we know τ satisfies

$$Q_{\tau+1} + \tau b \geq B := Tb.$$

It immediately implies

$$\mathbb{E}[T - \tau] \leq \frac{1}{b} \mathbb{E}[Q_{\tau+1}] \leq \frac{1}{b} Q_{\max},$$

which provides the regret of the selection phase $\mathbb{E}[T - \tau] = \tilde{O}(T^{\frac{3}{4}}U^{\frac{1}{4}}/b)$ as Lemma 5.4.

B.3 Proof Theorem 5.1

Combining the regret of the above two phases, we can obtain the total regret of our algorithm

$$\begin{aligned}
& \text{Regret}(T) \leq v^* \mathbb{E}[T - \tau] + \text{Regret}(\tau) \\
& \leq \frac{G^2T}{2V} + \frac{T \ln |\mathcal{A}|}{\eta} + \frac{T}{\gamma} + \gamma U + \left(\frac{T}{V\gamma} + \frac{\gamma U}{V} + \frac{v^*}{b} \right) Q_{\max} \\
& \leq \frac{G^2}{2} \sqrt{\frac{T}{U}} + \ln |\mathcal{A}| \sqrt{T} + 2\sqrt{TU} + \left(2 + \frac{v^*}{b} \right) Q_{\max} \\
& \leq \frac{G^2}{2} \sqrt{\frac{T}{U}} + \ln |\mathcal{A}| \sqrt{T} + 2\sqrt{TU} + \left(2 + \frac{v^*}{b} \right) C_1 T^{\frac{3}{4}}U^{\frac{1}{4}},
\end{aligned}$$

which completes the proof.

C Experiment Supplements

C.1 Detailed Description for predictors

KNN. We take the embedding of the query as input to train KNN predictors. To adapt KNN predictor into online update scenario, we train KNN predictors for each LLM separately, based on 200 initial data. During online routing, we update the feedback provided by the environment to the KNN predictor corresponding to the selected LLM. For hyperparameter, we choose $k = 100$.

K-Means. We have implemented an online update mechanism for the K-Means predictor. Initially, k cluster centers are trained based on 200 initial data. During the online routing process, these k cluster centers remain fixed. For each online feedback, the mean value of the corresponding cluster and selected LLM are updated.

Gradient Boosting. The input of the Gradient Boosting predictor differs from that of other predictors. In addition to the embedding of the query Emb_x , we also incorporate the embedding of the available LLM's description Emb_a , meaning that the input is formed by concatenating these embeddings, i.e., $[\text{Emb}_x, \text{Emb}_a]$. Both embeddings are 768-dimensional vectors, and thus the concatenated input has a dimensionality of 1536.

Roberta. The Roberta models used in SPROUT to estimate the reward and cost can be found in <https://huggingface.co/CARROT-LLM-Routing>.

C.2 Detailed Description for baseline methods

Fixed Cost-sensitivity Router. Given a query x , for each LLM a , the Fixed Cost-sensitivity Router predicts the reward $\hat{r}(x, a)$ and the cost $\hat{c}(x, a)$. Then the router computes score s_a and chooses the LLM with the maximum score. Somerstep et al. [28] defines $s_a = (1 - \mu)\hat{r}(x, a) - \mu\hat{c}(x, a)$ and Jitkritum et al. [18] defines $s_a = \hat{r}(x, a) - \lambda\hat{c}(x, a)$, where μ and λ are fixed parameters.

Primal-Dual Router. For N queries and M candidate LLMs, we define $r_{i,j}$ and $c_{i,j}$ as the reward and cost, respectively, for the i^{th} query when using the j^{th} LLM, where $i \in [N]$ and $j \in [M]$. For Primal-Dual Router, we employ K-means to do the classification, establishing K cluster set $C(k) \subset [N], k \in [K]$. For each cluster, we compute the averaged reward and cost for each candidate LLM $j \in [M]$, i.e. $\bar{r}(j, k) = \frac{\sum_{i \in C(k)} r_{i,j}}{|C(k)|}$, $\bar{c}(j, k) = \frac{\sum_{i \in C(k)} c_{i,j}}{|C(k)|}$, where $|C(k)|$ is the size of the set $C(k)$. By defining the policy for each cluster k as $\pi(\cdot | k)$, we solve

$$\begin{aligned} \max_{\pi} \quad & \sum_{k=1}^K \sum_{j=1}^M \frac{|C(k)|}{N} \pi(j | k) \bar{r}(j, k) \\ \text{s.t.} \quad & \sum_{k=1}^K \sum_{j=1}^M \frac{|C(k)|}{N} \pi(j | k) \bar{c}(j, k) \leq \frac{B}{N} \\ & \sum_{j=1}^M \pi(j | k) = 1, \forall k \in [K], \end{aligned}$$

where $\frac{|C(k)|}{N}$ estimates the probability of the query in cluster k and $\frac{B}{N}$ is the per-query budget. With the K-means classifier based on a training set of size 200, we apply the policy π in online routing. Note that Primal-Dual Router does not rely on predictors. Instead, it utilizes the actual value from the training set to solve the policy.

Conservation Router. For a specific prompt x and a candidate LLM a , we denote the predicted reward and cost as $\hat{r}(x, a)$ and $\hat{c}(x, a)$, respectively. At round t , the Conservative Router makes a conservative decision by selecting the LLM with the highest reward while its cost is not over the budget. That is

$$a_t = \arg \max \hat{r}(x_t, a) \quad \text{s.t.} \quad \hat{c}(x_t, a) < b_t,$$

where b_t is the remaining per-round budget. If there is no feasible LLM (all predicted cost is higher than b_t), the cheapest LLM will be selected.

Ratio Router. The Ratio Router simply chooses the LLM with the highest reward-cost ratio, i.e.

$$a_t = \arg \max \frac{\hat{r}(x_t, a)}{\hat{c}(x_t, a)}.$$

C.3 Selection of the Nearly Optimal Hyperparameter

In this section, we will discuss how to select the optimal hyperparameter in Section 6.2. We examine approximately optimal λ under the following two situations:

- Assumes that the initial distribution remains unchanged throughout the entire time horizon with $T = 25000$ and budget $B = 20$. This represents a plausible assumption, as in practice, one may empirically estimate the distribution from an initial set of samples to determine an appropriate cost-sensitivity parameter. However, this approach inherently overlooks potential distribution shifts over time.
- Assumes that the prior knowledge of distribution shifts is already known. This constitutes a highly optimistic assumption, as it effectively reduces the online process to an offline one to determine an appropriate user cost-sensitivity parameter from a global perspective. However, it's often infeasible in practical deployment scenarios.

To identify optimal parameters, we systematically evaluated λ over various ranges under all the above configurations. The experimental results are presented in Figure 4, which shows that the $\lambda \in \{120, 150\}$ are the nearly optimal values in the two configurations, and we employed them in the experiments in Section 6.2. These experimental results demonstrate the inherent difficulty in determining optimal parameters for traditional fixed cost-sensitivity methods, while simultaneously highlighting BARouter's superior capability to operate effectively without requiring parameter tuning.

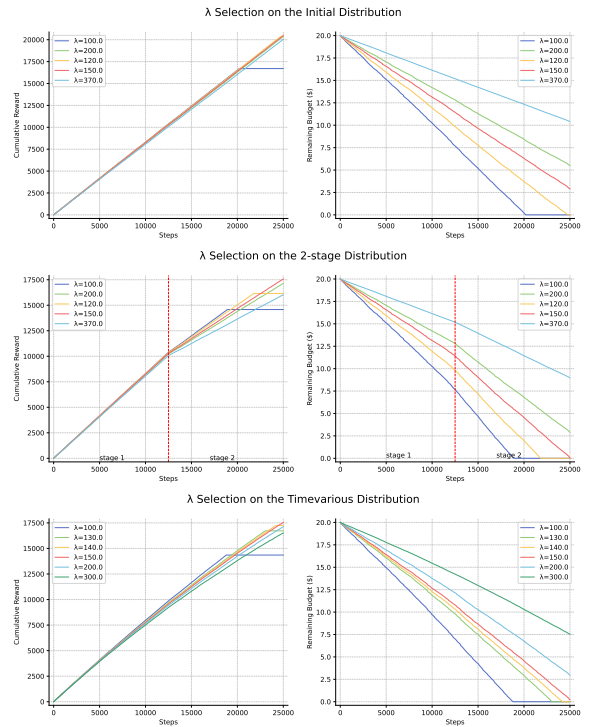


Figure 4: λ selection for non-stationary distribution.